Threat Hunt Report: Hunt 25 "Meridian"

Analyst: Ken Kariuki · **Date:** 26 September 2026 · **Result:** 16 of 16 flags, 1,675 / 1,675, completed on time
Workspace: LAW-HuntPractice (Azure Log Analytics, queried from Microsoft Defender Advanced Hunting)
Host: ip-10-1-15-67 (10.1.15.67, Ubuntu 22.04: Apache, PHP, MySQL) · **Attacker:** 10.1.134.57
Window: 2026-02-06 02:42:56 – 05:30:00 UTC (all times below are UTC, ordered by `EventTime_t`)

1. Summary

An external attacker (10.1.134.57) mapped Meridian's patient portal with four tools, got into the app's dashboard, and used a path-traversal flaw in `config_viewer.php` to read `/etc/passwd` and `database.conf`. Fifty-seven seconds after reading the config file, they logged in over SSH as the existing service account **svc_backup** with its password. The attacker could not use sudo or log in as root. They got root instead through a SUID copy of bash in `/tmp`, then planted a second SUID copy at **`/opt/meridian/scripts/health_check`**, where it ran as root and tried 73 times to reach **10.1.134.57:43212**. Patient data was exported through the web app at 03:56.

The host's automated defence stack reverted SSH keys, crontabs and temp directories, and removed `/tmp/rootbash`. It only **flagged** `health_check` and never removed it. The egress block stopped the beacon's connections but not the process. The stack's first sweep had also removed **sysmon.service** before the intrusion began, which left the incident without process-level telemetry. At 05:30 the returning admin had captured memory with AVML, and the root backdoor was still running (PID 267155).

The incident is not contained. `health_check` must be killed and deleted by hand, and svc_backup's credentials must be rotated (section 6).

2. Timeline

UTC	Event	Source
02:53:31 – 03:08:22	Defence stack bootstrap: detection burst, then first sweep; **sysmon.service removed** 03:08:19	Defender_CL
03:47:32	Recon starts: Nmap NSE (29 requests, plus 4 with no user agent)	Access_CL
03:52:09	curl, manual probing: `/`, `/robots.txt`, **`/config_viewer.php`** → **302** (03:52:29)	Access_CL
03:53:36 – 03:54:09	Login: POST `/index.php` 200, 200, then **302** → **`/dashboard.php`** **200**	Access_CL
03:54:54 / 03:55:05	WhatWeb fingerprint; gobuster directory brute force (18,454 requests)	Access_CL
03:56:28	**"Patient data export from 10.1.134.57"**	Defender_CL (WEB)
04:07:59	`config_viewer.php?file=../../../etc/passwd` → 200, 3,077 bytes	Access_CL
04:08:34	`config_viewer.php?file=database.conf` → 200, 1,231 bytes	Access_CL
04:09:31	**"Accepted password for svc_backup from 10.1.134.57"**, 57 seconds later	Auth_CL
04:15:07	`sudo -l` as svc_backup: "command not allowed"	Auth_CL
04:40:15	**`/tmp/rootbash -p`** executed, uid 1001, euid 0	Audit_CL (key tmp_exec)
04:44:25 – 04:46:52	Root SSH attempts from 10.1.15.67 and 10.1.134.57: "Failed password for root"	Auth_CL
04:48:28 – 05:02:17	73 × "Blocked exfil to 10.1.134.57:43212 from 10.1.15.67" (every ~11 s)	Defender_CL (NETWORK)
04:51	`health_check` process image created (its `/proc/<pid>/exe` link)	Snapshot_CL
05:05:01 – 05:05:06	Final sweep: kills and removes `/tmp/rootbash`, reverts **backup.conf**, only *flags* `health_check`	Defender_CL (SWEEP)

UTC	Event	Source
05:23:01	Admin `ubuntu` runs `sudo -i`	Auth_CL
05:27:48, 05:28:24	`/tmp/avml /tmp/evidence/memory.lime` (memory capture, twice)	Audit_CL
05:30:00	Snapshot: `health_check` PID 267155 still running, real uid 1001, effective uid 0	Snapshot_CL

3. Findings by kill-chain stage

Reconnaissance (T1595). Grouping the attacker's web requests by user agent gives four tools in order of first appearance: **nmap, curl, whatweb, gobuster**. The rows with no user agent arrive in the same second as Nmap and belong to it.

Initial access (T1190). Before the wordlist scan started, the attacker requested `config_viewer.php` on its own. After logging in, they used its unscoped `file` parameter for path traversal and read `/etc/passwd` (usernames), then `database.conf` (credentials, T1552.001). The access log does not record POST bodies, so it cannot show the login bypass itself. The briefing attributes the bypass to SQL injection.

Credential access (T1078). A password login as **svc_backup**, an account that already existed, 57 seconds after the config read. That is credential reuse, not brute force. Two escalation routes failed: **sudo** (svc_backup has no sudo rights) and **SSH as root** (wrong password, four attempts from inside and four from outside).

Privilege escalation (T1548.001). A pre-staged SUID bash copy, `/tmp/rootbash -p`, gave root at 04:40:15. The logs do not show how the binary got onto disk (see section 5).

Persistence (T1036.005, T1564.001). A second SUID copy, `/opt/meridian/scripts/health_check`, sits beside the legitimate root cron script `backup.sh` under a name that sounds like routine maintenance. It survived because the sweep only fixes SSH keys, crontabs, services and temp directories (including SUID files in /tmp). Its `scripts/` baseline check only detects: "Non-baseline file in scripts/: health_check" was logged and never acted on.

Command and control (T1071). `health_check` tried to reach **10.1.134.57:43212** 73 times on a schedule. The egress block stopped the connections, but blocking traffic is not neutralising: the process was never killed and would reconnect as soon as the block is lifted.

Collection (T1213, T1565.001). The defence stack logged a **patient data export** (patient medical records, PHI) at 03:56:28, two minutes after the dashboard login. The final sweep also caught and reverted a tampered **backup.conf**.

Defence evasion and detection gaps (T1562.001). The detection burst before 03:47 is a **false positive**: the stack detecting and reverting its own setup more than 50 minutes before any attacker activity. The real damage came from the stack's own first sweep, which removed **sysmon.service** as a "new systemd service". That took out process-creation, command-line and network-connection logging for the whole incident.

4. Key queries

```
// Recon toolchain: one row per user agent, in order of first appearance
MeridianAccess_CL
| where isnotempty(EventTime_t) and ClientIp_s == "10.1.134.57"
| summarize FirstSeen = min(EventTime_t), Requests = count() by UserAgent_s
| sort by FirstSeen asc
```

```
// Path traversal reads through config_viewer.php
MeridianAccess_CL
| where ClientIp_s == "10.1.134.57" and RequestUri_s has "config_viewer"
| project EventTime_t, RequestUri_s, StatusCode_s, ResponseSize_s
```

```
// Credential reuse and the failed root attempts
MeridianAuth_CL
| where EventTime_t between (datetime(2026-02-06 04:08:30) .. datetime(2026-02-06 04:50:00))
| where RawMessage_s has_any ("Accepted password", "Failed password", "command not allowed")
| project EventTime_t, ProcessName_s, RawMessage_s
```

```
// What the sweep fixed versus what it only logged
MeridianDefender_CL
| where EventCategory_s == "SWEEP"
| project EventTime_t, RawMessage_s
```

```
// The live backdoor at 05:30
MeridianSnapshot_CL
| project ProcBeaconExeLink_s, ProcBeaconName_s, ProcBeaconPid_s, ProcBeaconUid_s
```

```
// Responder activity
MeridianAudit_CL
| where exe_s == "/tmp/avml"
| project EventTime_t, Argv_s, cwd_s
```

Method notes:

- Every query filters and orders by `EventTime_t`. `TimeGenerated` is ingestion time (2026-08-30).
- Tables were reached from Defender Advanced Hunting as `workspace("LAW-HuntPractice").<Table>`.

5. Detection gaps and evidence limits

- **No process telemetry.** With Sysmon removed at 03:08, nothing records how `rootbash` and `health_check` were created, how the SUID bit was set, or what commands ran in the SSH sessions.
- **Narrow audit rules.** auditd had one execution rule (`tmp_exec`, /tmp only) and no file-creation or permission-change rules, so the planting of `health_check` outside /tmp was invisible.
- **No request bodies.** The access log cannot show the login injection or the exact export query.
- **Detection without remediation.** The sweep flagged `health_check` in every run after 04:51 and never removed it.

6. Response and recommendations

To close the incident (manual, now):

1. Kill `health_check` (PID 267155) and delete `/opt/meridian/scripts/health_check`. Then sweep the host for other SUID files: `find / -perm -4000 -newer /etc/hostname`.
2. Rotate svc_backup's password and the database credentials in `database.conf`. Review svc_backup's `authorized_keys`, and consider disabling interactive SSH for service accounts.
3. Keep the egress block on 10.1.134.57 in place, and block the attacker IP inbound.
4. Preserve `/tmp/evidence/memory.lime` (AVML) and analyse it for the beacon's configuration and any other implants.
5. Treat the patient data export as a reportable PHI breach and start the notification process.

To fix the gaps:

1. Patch `config_viewer.php`: restrict `file` to an allow-list, canonicalise paths, and refuse traversal.
2. Move secrets out of web-readable config files.
3. Restore Sysmon for Linux and add it to the sweep's service baseline, so the stack stops removing its own sensor.
4. Add auditd rules for `chmod` / `setuid` changes and for writes under `/opt/meridian/`.
5. Give the `scripts/` baseline check a remediation action (quarantine and kill), or alert a human the first time it fires.
6. Alert on the pattern this hunt followed: a sensitive file read over the web, then an SSH login within minutes from the same IP.

7. MITRE ATT&CK

T1595 Active Scanning · T1190 Exploit Public-Facing Application · T1552.001 Credentials in Files · T1078 Valid Accounts · T1548.001 Setuid and Setgid · T1036.005 Match Legitimate Name or Location · T1564.001 Hidden Files and Directories · T1071 Application Layer Protocol · T1213 Data from Information Repositories · T1565.001 Stored Data Manipulation · T1562.001 Impair Defenses: Disable or Modify Tools · T1005 Data from Local System (responder capture)